Computer Physics Communications 310 (2025) 109512

Contents lists available at ScienceDirect

Computer Physics Communications

journal homepage: www.elsevier.com/locate/cpc

Computer Programs in Physics

chemtrain: Learning deep potential models via automatic differentiation and statistical physics

Paul Fuchs [a,1], Stephan Thaler [a,b,1], Sebastien Röcken [a], Julija Zavadlav [a,c,*]

[a] Multiscale Modeling of Fluid Materials, Department of Engineering Physics and Computation, TUM School of Engineering and Design, Technical University of Munich, Germany
[b] Valence Labs, Montreal, QC, Canada
[c] Atomistic Modeling Center, Munich Data Science Institute, Technical University of Munich, Germany

ARTICLE INFO

The review of this paper was arranged by Prof. W. Jong

Dataset link: https://github.com/tummfm/chemtrain

Keywords:
Machine learning
Molecular dynamics
Coarse-graining
Many-body potential energy
Deep neural networks

ABSTRACT

Neural Networks (NNs) are effective models for refining the accuracy of molecular dynamics, opening up new fields of application. Typically trained bottom-up, atomistic NN potential models can reach first-principle accuracy, while coarse-grained implicit solvent NN potentials surpass classical continuum solvent models. However, overcoming the limitations of costly generation of accurate reference data and data inefficiency of common bottom-up training demands efficient incorporation of data from many sources. This paper introduces the framework `chemtrain` to learn sophisticated NN potential models through customizable training routines and advanced training algorithms. These routines can combine multiple top-down and bottom-up algorithms, e.g., to incorporate both experimental and simulation data or pre-train potentials with less costly algorithms. `chemtrain` provides an object-oriented high-level interface to simplify the creation of custom routines. On the lower level, `chemtrain` relies on JAX to compute gradients and scale the computations to use available resources. We demonstrate the simplicity and importance of combining multiple algorithms in the examples of parametrizing an all-atomistic model of titanium and a coarse-grained implicit solvent model of alanine dipeptide.

Program summary

Program Title: `chemtrain`
CPC Library link to program files: https://doi.org/10.17632/m6fxmcmfzz.1
Developer's repository link: https://github.com/tummfm/chemtrain
Licensing provisions: Apache-2.0
Programming language: python
Nature of problem: Neural Network (NN) potentials provide the means to accurately model high-order many-body interactions between particles on a molecular level. Through linear computational scaling with the system size, their high expressivity opens up new possibilities for efficiently modeling systems at a higher precision without resorting to expensive, finer-scale computational methods. However, as common for data-driven approaches, the success of NN potentials depends crucially on the availability of accurate training data. Bottom-up trained state-of-the-art models can match ab initio computations closer than their actual accuracy but can still predict deviations from experimental measurements. Including more accurate reference data can, in principle, resolve this issue, but generating sufficient data is infeasible even with less precise methods for increasingly larger systems. Supplementing the training procedure with more data-efficient methods can limit required training data [1]. In addition, the models can be fully or partially trained on macroscopic reference data [2,3]. Therefore, a framework supporting a combination of multiple training algorithms could further expedite the success of NN potential models in various disciplines.
Solution method: We propose a framework that enables the development of NN potential models through customizable training routines. The framework provides the top-down algorithm Differentiable Trajectory Reweighting [2] and the bottom-up learning algorithms Force Matching [1] and Relative Entropy Minimization [1]. A high-level object-oriented API simplifies combining multiple algorithms and setting up sophisticated training

* Corresponding author at: Multiscale Modeling of Fluid Materials, Department of Engineering Physics and Computation, TUM School of Engineering and Design, Technical University of Munich, Germany.
E-mail address: julija.zavadlav@tum.de (J. Zavadlav).
[1] Contributed equally.

https://doi.org/10.1016/j.cpc.2025.109512
Received 28 August 2024; Received in revised form 10 January 2025; Accepted 20 January 2025

Available online 28 January 2025
0010-4655/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

routines such as active learning. At a modularly structured lower level, the framework follows a functional programming paradigm relying on the machine learning framework JAX [4] to simplify the creation of algorithms from standard building blocks, e.g., by deriving microscopic quantities such as forces and virials from any JAX-compatible NN potential model and scaling computations to use available resources.

References

[1] S. Thaler, M. Stupp, J. Zavadlav, Deep coarse-grained potentials via relative entropy minimization, J. Chem. Phys. 157 (24) (2022) 244103, https://doi.org/10.1063/5.0124538.
[2] S. Thaler, J. Zavadlav, Learning neural network potentials from experimental data via Differentiable Trajectory Reweighting, Nat. Commun. 12 (1) (2021) 6884, https://doi.org/10.1038/s41467-021-27241-4.
[3] S. Röcken, J. Zavadlav, Accurate machine learning force fields via experimental and simulation data fusion, npj Comput. Mater. 10 (1) (2024) 1–10, https://doi.org/10.1038/s41524-024-01251-4.
[4] R. Frostig, M. J. Johnson, C. Leary, Compiling machine learning programs via high-level tracing.

1. Introduction

Molecular Dynamics (MD) is an essential tool in multiple disciplines, ranging from computational biology to mechanical engineering [1–3]. Classical force fields [4–6] concurrently developed with established MD software [7–10] offer inexpensive but accurate modeling of molecular systems in specialized fields. However, the simplicity of classical force fields prohibits modeling quantum effects such as charge transfer and bond-breaking. At the same time, available computational resources limit the possible system size and simulation time even for very simple classical atomistic force fields. Therefore, multiscale modeling approaches are crucial to extending MD simulation to systems with large scale separation [11–14].

Recently, Neural Networks (NNs) potential models emerged as a flexible approach to supplement or replace classical multiscale simulation techniques. NN potential architectures [15–21] model high-order multibody interactions using pre-defined [22,16] or learned [17–21] features of the local environment of the atoms. Therefore, the computational effort of employing NN-based force fields scales linearly with the size of the system, similar to classical force fields. At the atomistic scale, the NN models can predict energies and forces with a precision similar to first-principle methods [16,23,18]. Consequently, NN-enhanced MD can generate trajectories with similar accuracy but is less computationally expensive than ab initio MD [24] and introduces electronic quantum effects to the whole system, unlike hybrid quantum-mechanical / molecular-mechanical modeling [25]. In coarse-grained (CG) MD approaches, NN potentials can approximate important many-body interactions of the potential of mean force often neglected in classical CG potential models [26]. Therefore, NN potentials also enable highly accurate mesoscopic modeling, surpassing, e.g., classical implicit solvation models [27,28].

NN potential models are data-driven approaches for which optimal parametrizations must be inferred from experiments or simulations. The parameterization of potential models is typically categorized into top-down and bottom-up strategies [29]. Bottom-up approaches [30–32] learn parameters by fitting predictions of the model to large amounts of high-resolution data from a finer-scaled reference model. Top-down approaches [33–37] adjust the model such that simulations reproduce experimentally determined macroscopic properties. The majority of software packages supporting the training of NN-based potential models focus on bottom-up strategies [38–54] by matching potential energies, forces, and other microscopic quantities via the Force Matching (FM) algorithm [31]. With sufficient reference data, bottom-up trained state-of-the-art potential can match the accuracy of the data generating (e.g., density functional theory) method [16,23,18]. Nevertheless, the models cannot surpass the method's accuracy, affecting the quality of macroscopic predictions [55,56]. Including reference data from more precise (e.g., coupled cluster) methods can alleviate this issue [23]. Still, the investigation of increasingly larger systems, particularly when developing CG models, even renders the generation of sufficient data with less precise methods computationally infeasible. One option is to limit required reference data by supplementing the training procedure with more data-efficient, albeit more expensive, methods such as Relative Entropy Minimization (RM) for CG systems [28]. More generally, a viable alternative to enforce consistency with macroscopic observables for models at all scales is to partially [57,55] or fully [58,59] employ top-down learning. Classically, top-down learning approaches [34,33,36,37] are focused on training potential models with particular functional forms, but can be generalized to support a wider range of macroscopic observables and potential models through reweighting techniques [35,60–62,58,63]. Particularly, the Differentiable Trajectory Reweighting (DiffTRe) algorithm [58] generalizes reweighting based top-down learning to NN potential models by leveraging Automatic Differentiation (AD) and differentiable physics MD packages [43,64,65], available for many established machine learning libraries [66–68]. Additionally, the DiffTRe algorithm enables the implementation of reweighting-based RM as a more data-efficient alternative or complement to FM to learn CG NN potential models. However, available software [63,43] only provides a DiffTRe-like method with support for classical force fields or adjusts NN potential models through minimally-biasing methods [36,37], which correct macroscopic properties by adding a minimal correction to the NN potential model, but are not suitable for learning transferable potential models [56].

In this paper, we propose `chemtrain`, a framework that enables the development of sophisticated NN potentials through highly customizable training routines. As elementary algorithms, `chemtrain` provides the well-known bottom-up approach of FM [28] and its uncertainty-aware probabilistic extension [69] through an interface to JaxSGMC [70], the bottom-up approach of RM [28] and the top-down approach DiffTRe [58]. An object-oriented high-level Application Programming Interface (API) simplifies the creation of custom training procedures, e.g., by combining algorithms [57] or employing active learning strategies [71]. At the lower level, `chemtrain` is modularly structured, following a functional paradigm to leverage JAX [68] transformations to simplify setting up algorithms from basic building blocks. These transformations enable, e.g., the automatic derivation of microscopic quantities, such as forces and virials, from any JAX-compatible NN potential models and the parallelization of computations across multiple devices to efficiently use available resources. Hence, `chemtrain` aims to provide a simple, flexible, and computationally efficient software solution to develop sophisticated potential models for various fields.

This paper is structured as follows: To bridge between users with machine learning and physics backgrounds, we provide a reiteration of the fundamentals of MD in section 2 and the foundation of top-down/bottom-up training algorithms in section 3, respectively. In section 4, we explain the design of `chemtrain` and provide a detailed overview of the structure, including a summary of the most important building blocks. Section 5 outlines the importance and usage of `chemtrain` at two relevant examples combining training algorithms to efficiently learn improved CG and atomistic potential models. Finally, we present possibilities for the future of `chemtrain` in section 6.

2. Molecular dynamics and statistical mechanics

In the following section, we briefly reiterate the fundamentals of MD [72] as a computational tool to model the time evolution of a molecular system. Under the assumption of ergodicity, we outline a connection between the microscopic long-term simulation of a single system and the macroscopic probabilistic statistical mechanics perspective on many identical but independent systems. This connection forms a basis for the variational approaches to top-down and bottom-up learning of potential models in section 3.

MD aims to simulate material properties at a molecular level by solving the classical equations of motion for a system of point-mass-like particles [72]. For a system of N particles with positions $\boldsymbol{r} = (\boldsymbol{r}_1, \dots, \boldsymbol{r}_N) \in \mathbb{R}^{3N}$ and momenta $\boldsymbol{p} = (\boldsymbol{p}_1, \dots, \boldsymbol{p}_N) \in \mathbb{R}^{3N}$, Newtons second law describes the evolution of the system via $6N$ coupled differential equations

$$\dot{\boldsymbol{r}}_i = \frac{\boldsymbol{p}_i}{m_i}, \quad \dot{\boldsymbol{p}}_i = \boldsymbol{F}_i(\boldsymbol{r}, \boldsymbol{p}_i), \tag{1}$$

where m_i denotes the mass of the i-th particle. The forces $\boldsymbol{F}$ model all interactions between the particles and can even account for non-conservative interactions with the environment. However, the dynamics of an isolated piece of matter must obey to the conservation of energy. Therefore, one typically restricts the interaction forces to be conservative by defining a scalar potential $U(\boldsymbol{r})$ such that

$$\boldsymbol{F}_i(\boldsymbol{r}) = -\frac{\partial U(\boldsymbol{r})}{\partial \boldsymbol{r}_i}. \tag{2}$$

Combined with the kinetic energy K for simple point masses

$$K(\boldsymbol{p}) = \sum_{i=1}^{N} \frac{|\boldsymbol{p}_i|^2}{m_i}, \tag{3}$$

the scalar potential forms the Hamiltonian $H(\boldsymbol{r}, \boldsymbol{p}) = K(\boldsymbol{p}) + U(\boldsymbol{r})$ equal to the total energy of the system. Many sophisticated numerical schemes exist for solving the ordinary Newtonian or Hamiltonian equations. However, integrating the Hamiltonian equations with a symplectic integrator bounds the error of energy conservation and enables stable simulations of molecular systems over very long time spans.

MD models matter at a microscopic level but aims to reproduce and understand experiments at macroscopic scales. Therefore, it is necessary to interpret the microscopic results from the macroscopic perspective of statistical mechanics and thermodynamics [72]. The basis of statistical mechanics relies on the concept of ensembles. An ensemble $\mathcal{E}$ is a collection of all the identical systems that reproduce a set of macroscopic observables. At time t, the microscopic states $\boldsymbol{x}_\lambda$ of the systems $\lambda \in \mathcal{E}$ are distributed in phase space according to a distribution function $p(\boldsymbol{x}, t)$. Therefore, a macroscopic observable A obtained as the average of instantaneous microscopic properties $a(\boldsymbol{x}_\lambda)$ for all ensemble members $\lambda \in \mathcal{E}$ is equivalently given as an expectation on the ensemble distribution function

$$A(t) = \frac{1}{|\mathcal{E}|} \sum_{\lambda \in \mathcal{E}} a(\boldsymbol{x}_\lambda) = \langle a(\boldsymbol{x}) \rangle_{p(\boldsymbol{x},t)} = \int a(\boldsymbol{x}) p(\boldsymbol{x}, t) d\boldsymbol{x}. \tag{4}$$

Since the microscopic states of the systems evolve according to the same dynamics $\dot{\boldsymbol{x}} = \xi(\boldsymbol{x})$, the ensemble distribution function $p(\boldsymbol{x}, t)$ is directly linked to the dynamics [73]. However, practically exploiting this link through MD relies on the assumption of ergodicity and thermodynamic equilibrium $p(\boldsymbol{x}, t) = p(\boldsymbol{x})$. Generally, two systems in an ensemble might reside in separate regions of phase space and never visit the same states. However, a system in an ergodic ensemble will evolve through all points in phase space with a frequency proportional to $p(\boldsymbol{x})$ if given an infinite amount of time. Therefore, the long-term average over the evolution of a such single system

$$A = \langle a(\boldsymbol{x}) \rangle_\tau = \lim_{t \to \infty} \frac{1}{t} \int_0^t a(\boldsymbol{x}(\tau)) d\tau \tag{5}$$

approaches the phase-space average over all accessible microstates. Hence, a single MD simulation suffices to predict the equilibrium observables of an ergodic ensemble.

A commonly employed ensemble for MD is the canonical ensemble of N particles in a space of constant volume, maintaining an average temperature T via thermal exchange with an environment. In the canonical ensemble, the position variables $\boldsymbol{r}$ are independent of the momenta $\boldsymbol{p}$, leading to a reduced form of the equilibrium distribution

$$p_U(\boldsymbol{r}) = e^{-\beta U(\boldsymbol{r}) - \beta Q_U}, \quad Q_U = -\frac{1}{\beta} \log \int e^{-\beta U(\boldsymbol{r})} d\boldsymbol{r}, \tag{6}$$

where $\beta = \frac{1}{k_B T}$ is the inverse of the microscopic temperature $k_B T$ and k_B is the Boltzmann constant. While statistical mechanics arrive at $p(\boldsymbol{x})$ by marginalizing out degrees of freedom of an excessively larger thermal bath, simulation of these degrees of freedom is inconvenient. Instead, in practical MD one perturbs the Hamiltonian dynamics of the system, e.g., by adding a chained Nosé-Hoover thermostat [74] or performing stochastic Langevin Dynamics [75] such that the new dynamics $\xi(\boldsymbol{x})$ lead to a quasi-canonical equilibrium distribution.

3. Machine learning of potential models

3.1. Top-down learning

Top-down learning aims to find a potential model U such that the molecular system reproduces a set of macroscopic targets $\{\hat{A}_k\}$. In a variational formulation, the optimal potential parameters θ minimize the difference $\Delta_k = \hat{A}_k - \mathcal{A}_k[U_\theta]$ between the target values $\hat{A}_k$ and simulated predictions $\mathcal{A}_k[U_\theta]$ through a weighted sum of errors

$$\mathcal{L}(\theta) = \sum_{k=1}^{K} \gamma_k \ell_k(\Delta_k), \tag{7}$$

where γ_k weights per-target contributions based on, e.g., the mean-squared error $\ell(\Delta_k) = \frac{1}{2 \dim \Delta_k} \|\Delta_k\|^2$. We introduced $\mathcal{A}$ to conveniently account for macroscopic observables consisting of multiple ensemble averages, such as the heat capacity [76].

Gradient-based optimization is a widely used and efficient method of training NN models. At first glance, implementing the computation of $\mathcal{A}_K[U_\theta]$ in a differentiable MD framework provides the necessary gradients of the loss via AD. However, molecular systems often exhibit chaotic dynamics [77], such that small perturbations of the system can critically affect the solution $\boldsymbol{x}(t)$ of the differential equations. Consequently, employing AD to differentiable simulations does not guarantee meaningful gradients [78], which are even susceptible to the exploding gradients issue [79,58] and costly for many integration steps. Therefore, MD simulations are suitable to efficiently predict the value of the loss but can be insufficient for gradient-based optimization [80,58].

From the probabilistic perspective of thermodynamic perturbation theory [81], a change in potential from $\tilde{U}$ to $U = \tilde{U} - \Delta U$ affects the relative probability $p_U / p_{\tilde{U}}$ of conformations $\boldsymbol{r}$. Interpreting this relative probability as weighting function $w_U = p_U / p_{\tilde{U}}$ establishes a connection between the ensemble averages of the perturbed and unperturbed system

$$A_U = \langle a(\boldsymbol{r}) \rangle_{p_U} = \int a(\boldsymbol{r}) w_U(\boldsymbol{r}) p_{\tilde{U}}(\boldsymbol{r}) d\boldsymbol{r} = \langle w_U(\boldsymbol{r}) a(\boldsymbol{r}) \rangle_{p_{\tilde{U}}}, \tag{8}$$

as long as both distribution functions overlap. The canonical ensemble fulfills the overlap criterion for any finite perturbation ΔU with a corresponding weight function

$$w_U(\boldsymbol{r}) = \frac{e^{-\beta \Delta U(\boldsymbol{r})}}{\left\langle e^{-\beta \Delta U(\boldsymbol{r})} \right\rangle_{p_{\tilde{U}}}}. \tag{9}$$

Assuming that a parameter dependence of U is introduced only through ΔU_θ, the weighted ensemble average leads to a well-defined expression for the observable gradients

$$\frac{\partial A_U}{\partial \theta} = \left\langle \frac{\partial a(\theta, \boldsymbol{r})}{\partial \theta} w_U(\theta, \boldsymbol{r}) + a(\theta, \boldsymbol{r}) \frac{\partial w_U(\theta, \boldsymbol{r})}{\partial \theta} \right\rangle_{p_{\tilde{U}}} . \tag{10}$$

The direct application of thermodynamic perturbation theory is often analytically intractable as it requires the evaluation of ensemble averages given in equations (8) and (10). However, combined with Monte Carlo (MC) or MD as in the Umbrella sampling method [82], perturbation theory provides a basis for viable approaches to top-down learning [35,60,58,63].

One particularly flexible approach is the recently proposed DiffTRe algorithm [58]. Starting the optimization from an initial guess $U^{(0)}$, the algorithm draws decorrelated samples $\{\boldsymbol{r}^{(i)}\}_{i=1}^{D}$ from $\tilde{U} \leftarrow U^{(0)}$ via MD simulations. Relying on a fully-differentiable implementation of a_θ, ΔU_θ, and the reweighting procedure, the algorithm leverages AD to update θ based on the gradients of the loss (7). To account for the statistical error from approximating the exact perturbation formula (8) with a finite number of samples [83], the algorithm measures the effective sample size [61]

$$\mathrm{ESS} = e^{-\sum_{i=1}^{D} w_U^{(i)} \log w_U^{(i)}} \in [1, D] \tag{11}$$

and resamples new reference states from the current potential candidate $\tilde{U} \leftarrow U^{(i)}$ whenever the ESS drops below a pre-defined threshold. Therefore, the DiffTRe algorithm enables training expressive potential models with complicated functional forms, such as NN potential models, based on equilibrium macroscopic observables. Additionally, an extension to dynamic properties is, in principle, also possible [80].

3.2. Bottom-up learning

We approach the problem of bottom-up learning from the perspective of coarse-graining from a fine-grained (FG) to a coarse-grained (CG) particle system. A typical example is modeling an FG atomistic representation with a coarser approximation, in which several atoms are grouped into effective beads modeled as single particles. However, learning effective atomistic models from ab initio data also complies with this idea [84].

A common first step in coarse-graining is to link the CG system to its FG reference by selecting a mapping $\mathcal{M} : \mathbf{R}^{3n} \to \mathbf{R}^{3N}$. This mapping relates the positions of the N CG particles $\boldsymbol{R}$ to the positions of the n fine-grained particles $\boldsymbol{r}$ via

$$\boldsymbol{R} = \mathcal{M}(\boldsymbol{r}). \tag{12}$$

If the FG and CG models act on the same scale, e.g., in training an atomistic model on ab initio reference data, the identity function is a trivial choice for the mapping. In coarse-graining, a computationally convenient choice of mapping places the position of the CG particle at the center of mass of the fine-grained particles that constitute an effective bead [84]. However, more sophisticated mappings, e.g., based on dimensionality reducing NNs [85] can also be useful.

3.2.1. Force matching

The FM approach was originally proposed as an efficient method to learn atomistic potentials from large amounts of first-principle data [31]. However, it is also a practical and theoretically well-founded approach to couple particle-based models of different scales [84,32].

The motivation for employing FM as a coarse-graining approach lies in the uniqueness of forces of a thermodynamic consistent model [32]. A CG model U^{CG} is consistent with the reference model U^{FG} and the mapping $\mathcal{M}$, if it generates CG conformations $\boldsymbol{R} \sim p^{\mathrm{CG}}(\boldsymbol{R})$ that match the mapped samples $\mathcal{M}(\boldsymbol{r})$ from the fine-grained reference $\boldsymbol{r} \sim p^{\mathrm{FG}}(\boldsymbol{r})$. More formally, the distribution p^{CG} of the consistent model U^{CG} is

$$p^{\mathrm{CG}}(\boldsymbol{R}) = \langle \delta [\mathcal{M}(\boldsymbol{r}) - \boldsymbol{R}] \rangle_{p^{\mathrm{FG}}} . \tag{13}$$

From equations (2) and (6) follows that the consistent distribution uniquely defines the forces $\boldsymbol{F}$ acting on the CG sites $\boldsymbol{R}$ as

$$\boldsymbol{F}_I(\boldsymbol{R}) = -\frac{1}{\beta} \frac{\partial}{\partial \boldsymbol{R}_I} \log p^{\mathrm{CG}}(\boldsymbol{R}). \tag{14}$$

Under the assumption of a linear mapping $\mathcal{M}(\boldsymbol{R}) = \boldsymbol{C}\boldsymbol{R}$ and a linear mapping of forces $\mathcal{F}(\boldsymbol{f}) = \boldsymbol{D}\boldsymbol{f}$, both with weak constraints, it is possible to prove that these forces are the unique minimum of the functional [32]

$$\chi^2_{\boldsymbol{F}}[\boldsymbol{F}] = \frac{1}{3N} \sum_{I=1}^{N} \left\langle \left\| \mathcal{F}_I(\boldsymbol{f}(\boldsymbol{r})) - \boldsymbol{F}_I(\mathcal{M}(\boldsymbol{r})) \right\|^2 \right\rangle_{p^{\mathrm{FG}}} . \tag{15}$$

Therefore, minimizing the functional for a parameterized model $\tilde{\boldsymbol{F}}_\theta = -\frac{\partial U_\theta}{\partial \boldsymbol{R}_I}$ provides a well-based variational approach to coarse-graining.

An additional step is necessary to apply FM practically. Analogous to top-down coarse-graining, computing the exact value of the residual in equation (15) is analytically intractable. Instead, one must resort to estimates based on a finite number of samples $\mathcal{D} = \left\{ \left(\mathcal{F}^{(i)}, \boldsymbol{R}^{(i)} \right) \right\}_{i=1}^{D}$ from the FG reference system, e.g., via MC or MD simulations. Then, it is possible to employ classical numerical optimization procedures [31,86] or stochastic gradient-based methods [87] to minimize the approximate loss function

$$\mathcal{L}_{\boldsymbol{F}}(\theta) = \frac{1}{3ND} \sum_{i=1}^{D} \sum_{I=1}^{N} \left\| \mathcal{F}_I^{(i)} - \boldsymbol{F}_I\left(\theta, \boldsymbol{R}^{(i)}\right) \right\|^2 . \tag{16}$$

For linear models with infinite data, a unique optimal model closest to the consistent model exists [32]. However, training more complex models on inadequately sampled data introduces uncertainty with possibly severe consequences on the robustness of the learned models [88]. By employing uncertainty-aware inference methods, it is possible to account for the effects of finite data size and model pluralism. For example, adopting the FM loss of equation (16) as a likelihood into a probabilistic formulation

$$p(\theta|\mathcal{D}) \propto p(\theta) \prod_{k=1}^{D} \mathcal{N}\left(\mathbf{F}_\theta\left(\boldsymbol{R}^{(k)}\right) \mid \mathcal{F}^{(k)}, \sigma^2 \mathbb{I} \right) \tag{17}$$

enables the application of Bayesian Inference. However, especially for neural network models, exact Bayesian Inference, e.g., via stochastic-gradient MC [89] is costly [69]. Hence, approximate or non-Bayesian schemes such as Drop-out MC [90] or the Deep Ensemble method [91, 69] are popular alternatives.

Training only a single but accurate model is desirable for most practical applications. At the atomistic scale, additional training on other potential-dependent properties [31] increases available information and improves accurate predictions of these properties. Typical examples are potential energy, which provides information about the relative probability of samples compared to the exclusive gradient information from forces, and virials, which are important, e.g., for NPT simulations. In principle, matching these additional quantities is also possible for FM in the CG setting [92]. However, thermodynamic observables at the atomistic level are generally not directly applicable at the CG scale [93], complicating a simple extension to arbitrary quantities. Additionally, the extended approach enables the training of models that couple the prediction of forces for MD with the prediction of non-classical properties, e.g., electronic properties [94]. Hence, a more general formulation of the FM approach at the atomistic scale matches additional parameter-dependent quantities a_k to targets $\hat{a}_k$ by minimizing a weighted sum of losses

$$\mathcal{L}(\theta) = \gamma_{\boldsymbol{F}} \mathcal{L}_{\boldsymbol{F}}(\theta) + \frac{1}{D} \sum_{i=1}^{D} \sum_{k=1}^{K} \gamma_k \left\| \hat{a}_k^{(k)} - a_k\left(\theta, \boldsymbol{R}^{(k)}\right) \right\|^2 , \tag{18}$$

where γ_k weights the contributions of the per-target losses, e.g., $\mathcal{L}_{\boldsymbol{F}}$, to the total loss.

3.2.2. Relative entropy minimization

The relative entropy, also known as Kullback-Leibler divergence [95], is a classical information-theoretical measure of the similarity of two distributions. Descriptively, minimizing the relative entropy maximizes the

likelihood of obtaining a sample from a target distribution when drawn from an independent reference distribution [30]. Hence, RM provides an alternate foundation for learning good approximations of molecular systems. For a reference distribution $p^{\mathrm{CG}}(\boldsymbol{r})$ from a surrogate model, the relative entropy towards a target distribution $p^{\mathrm{FG}}(\boldsymbol{r})$ is

$$S_{\mathrm{rel}} = \int p^{\mathrm{FG}}(\boldsymbol{r}) \log \frac{p^{\mathrm{FG}}(\boldsymbol{r})}{p^{\mathrm{CG}}(\boldsymbol{r})} d\boldsymbol{r} \geq 0. \tag{19}$$

In the context of coarse-graining, the parametrizable model $p_\theta^{\mathrm{CG}}(\boldsymbol{R})$ does not act on the fine-grained sites $\boldsymbol{r}$ but on the CG sites $\boldsymbol{R}$ connected through the mapping $\mathcal{M}$. Without any prior assumption, any FG conformation $\boldsymbol{r}$ complying with a CG conformation $\boldsymbol{R}$ is equally likely, i.e., $p(\boldsymbol{r} \mid \boldsymbol{R}) \propto \delta(\mathcal{M}(\boldsymbol{R}) - \boldsymbol{r})$. Thus, by the law of total probability, the CG model defines the probability of FG conformations as

$$p^{\mathrm{CG}}(\boldsymbol{r}) = \int p(\boldsymbol{r} \mid \boldsymbol{R}) p_U^{\mathrm{CG}}(\boldsymbol{R}) d\boldsymbol{R} = \frac{p_U^{\mathrm{CG}}(\mathcal{M}(\boldsymbol{r}))}{\Omega_{\mathcal{M}}(\mathcal{M}(\boldsymbol{r}))}, \tag{20}$$

where Ω ensures the proper normalization of the conditional probability. Therefore, the relative entropy (19) between a canonical FG and canonical CG system is

$$S_{\mathrm{rel}} = \beta \left\langle U^{\mathrm{CG}}(\mathcal{M}(\boldsymbol{r})) - U^{\mathrm{FG}}(\boldsymbol{r}) \right\rangle_{\mathrm{FG}} - \beta(Q_U^{\mathrm{CG}} - Q^{\mathrm{FG}}) + S_{\mathrm{map}}, \tag{21}$$

with an irreducible contribution $S_{\mathrm{map}} = \langle \log \Omega_{\mathcal{M}}(\mathcal{M}(\boldsymbol{r})) \rangle_{p^{\mathrm{FG}}}$ arising from the degeneracy of the mapping.

In general, the relative entropy is hard to estimate since the mapping entropy S_{map}, and the free energies Q^{FG} and Q_U^{CG} are not simple ensemble averages. However, S_{map} and Q^{FG} are irrelevant for gradient-based optimization since they depend only on the mapping or the FG reference distribution, respectively, but are constant with respect to the parameters θ. Analogously, the absolute free energy Q_θ^{CG} has the same gradients as a free energy difference $\Delta Q_\theta^{\mathrm{CG}}$ with respect to a fixed reference potential. Unlike for the absolute free energy, free energy perturbation methods [61,81] enable the estimation of $\Delta Q_\theta^{\mathrm{CG}}$ trough ensemble averages. Therefore, the alternative loss function

$$\mathcal{L}_{\mathrm{RE}}(\theta) = \beta \left\langle U_\theta^{\mathrm{CG}}(\mathcal{M}(\boldsymbol{r})) \right\rangle_{\mathrm{FG}} - \beta \Delta A_\theta^{\mathrm{CG}} \tag{22}$$

consists only of ensemble averages and differs from the relative entropy only by a constant. Hence, minimizing $\mathcal{L}$ with respect to θ is equivalent to minimizing the relative entropy S_{rel}. In a practical implementation, the first summand of the loss can be estimated based on a finite number of samples from the FG system, e.g., like in the FM algorithm. For the second term, the DiffTRe algorithm provides a fully differentiable approach to compute perturbation-based estimates of the free energy difference [55].

3.3. Δ-learning approach

Adequate sampling of the conformational space is essential to the success of the presented learning methods. However, data obtained through MD simulations is biased, resulting in infrequently sampling of some regions. For example, strong repulsive forces at short intermolecular distances encode the Pauli exclusion principle but cause infrequent sampling of conformations with close particles. From only a few of these samples, it is difficult to learn these short-range forces via bottom-up learning methods [96]. Analogously, top-down learning via the DiffTRe method requires sufficient overlap between the distributions at subsequent reweighting steps. Consequently, sampling irrelevant conformations from an initial potential that does not model crucial properties of the interactions can severely impede learning success. Hence, a popular choice for the potential model in classical coarse-graining [97,96] and neural-networks-based learning [98,58,28] includes known properties through an additive prior potential

$$U(\theta, \boldsymbol{r}) = \Delta U(\theta, \boldsymbol{r}) + U^{\mathrm{Prior}}(\boldsymbol{r}). \tag{23}$$

The parametric model only learns the difference to the prior potential, leading to the Δ-learning terminology [99]. Especially for learning NN potential models, classical atomistic or CG force fields and functional forms provide a computational affordable basis for choosing a suitable prior potential [27,58,28].

4. Software

We developed `chemtrain` to advance the application of powerful potential models by facilitating the training of models via popular algorithms and promoting the development of new and elaborate training strategies. We structured `chemtrain` into two layers to accomplish both objectives: At the lower level, we designed `chemtrain` following a functional paradigm, using modular building blocks to create algorithms. At the higher level, an object-oriented API simplifies using and combining algorithms. The next section outlines the rationale behind the structure and design of `chemtrain` and the API. Then, we provide a more detailed description of the individual modules and implemented functionalities.

4.1. Software architecture

We outline the structure of `chemtrain` at the example of a Maximum Likelihood Estimation (MLE) algorithm. Learning a single optimal model maximizing a likelihood or minimizing a loss function, respectively, is the standard scenario for the RM, FM, and DiffTre algorithms (section 3) and easily extensible to an uncertainty-aware perspective, e.g., via the deep-ensemble approach [69,100]. Illustrated in Fig. 1, learning such a single-point model consists of three components: (1) Objectives such as the trainable potential model U_θ, reference systems, thermodynamic state and reference data. (2) Algorithm to evaluate the loss and loss gradients for the objectives, such as FM for microscopic and DiffTRe for macroscopic reference data. (3) Trainer, applying a numerical scheme to optimize the model parameters based on the loss.

An important part of `chemtrain` is efficient and modular implementations of basic training algorithms for the second component. The FM, RM, and DiffTRe algorithms differ in top-down and bottom-up training perspectives and the predicted quantities. Nevertheless, the algorithms perform similar routines in the training cycle: First, the algorithms load reference data (FM/RM) or sample new data from a reference system (RM/DiffTRe). Based on the sampled or loaded microscopic states, the algorithms compute instantaneous quantities $a(\cdot)$, e.g., forces (FM) and energies (FM/RM/DiffTRe). Finally, the algorithms compute a loss for the objectives using the reference data and microscopic predictions. This loss computation might involve the microscopic predictions directly (FM/RM) or a combination of multiple microscopic predictions, e.g., in the form of weighted ensemble averages (RM/DiffTRe). A computationally efficient implementation of these steps is crucial for the practical performance of the algorithms. Leveraging the described abstractions, multiple algorithms can benefit from carefully designed implementations. Therefore, `chemtrain` follows a modular structure, relying on scalable and flexible building blocks.

The building blocks follow a functional paradigm, relying on a functional implementation of the objectives in the third component. Therefore, the building blocks can leverage JAX [68] transformations to achieve scalability and flexibility. A good example of flexibility is the potential model. The jit-compilation of JAX enables the abstraction of the potential model into an `energy_fn_template`, which creates the model on the fly within the building blocks by applying a parametric architecture to a concrete system. Listing 1 provides an example of a Δ-learning potential model using the DimeNet++ [19] NN model and a classical force field as prior potential. The `jit`-compilation enables an efficient evaluation of this on-the-fly-created potential model and further derived quantities, e.g., forces. Moreover, the vectorization and parallelization capabilities of JAX enable an efficient evaluation of the potential model and other microscopic quantities on multiple

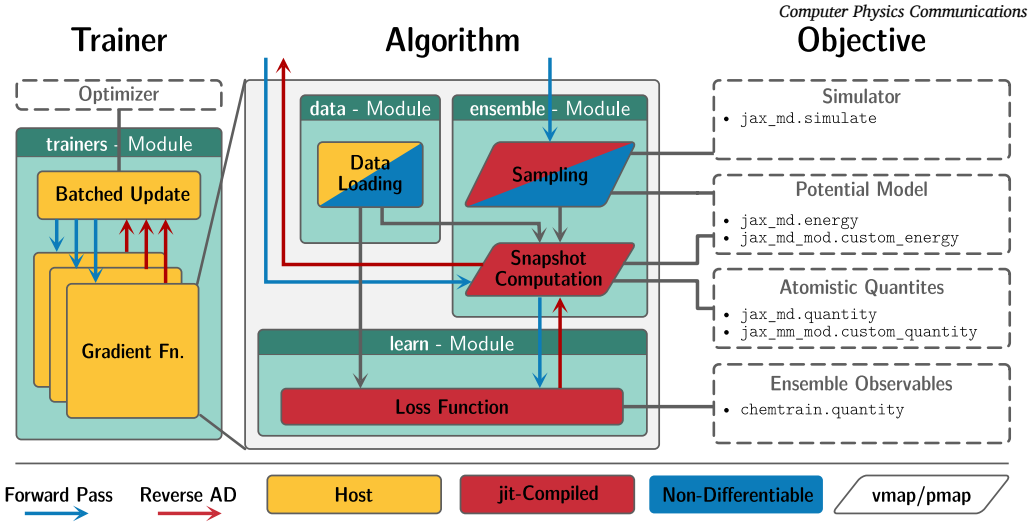


Fig. 1. Structure of a Maximum Likelihood Estimate (MLE) learning procedure in `chemtrain`. The trainer executes algorithms to perform gradient-based optimization of a single or multiple objectives. The algorithms assemble from multi-functional building blocks and rely on a modular definition of the objectives.

parallel samples. Therefore, `chemtrain` can scale the computation to use the full power of the devices and distribute workloads among multiple devices. Finally, the building blocks also benefit from the comprehensive AD functionalities of JAX. For example, the FM algorithm can derive a force model directly from the potential model by employing AD. Moreover, by deriving the parametric force model from the `energy_fn_template`, a second application of AD then yields the gradients of the FM loss function in equation (16). Hence, the functional implementation of building blocks enables a simple but scalable definition of objectives.

```
# Setup the graph NN based on the neighbor list
init_fn, gnn_energy_fn = neural_networks.dimenetpp_neighborlist(
    displacement_fn, r_cut, n_species, r_init, nbrs_init,
    embed_size=32, init_kwargs=mlp_init,
)

# Set up a prior potential, e.g., a classical force field
prior_energy_fn = prior_energy(topology, force_field)

# Flexible formulation of trainable potential model
def energy_fn_template(energy_params):

    # Enables obtaining the prior potential from the energy
    # function template
    if energy_params is None:
        return prior_energy_fn

    def energy_fn(pos, neighbor, **dynamic):
        # Dynamic quantities can contain the thermostat
        # temperature, e.g., to learn a temperature-dependent
        # CG model
        gnn_energy = gnn_energy_fn(
            energy_params, pos, neighbor, **dynamic
        )

        prior_energy = prior_energy_fn(pos, neighbor=neighbor)
        return prior_energy + gnn_energy
    return energy_fn
```

Listing 1: Defining a potential model as Δ-learning approach based on a prior potential and an NN potential. The functional return type enables other building blocks to apply JAX transformations, e.g., computing forces via AD.

In addition to the modular structure at the lower level, `chemtrain` provides an object-oriented API to facilitate its application to specialized problems. This API introduces the concept of trainer classes. A trainer class wraps single or multiple algorithms, interfacing the problem-specific objectives with the appropriate algorithm and the computation of gradient and loss information with numerical optimization. The interface between algorithms and objectives simplifies the usage of existing algorithms without exhibiting additional abstraction through modularity. Therefore, algorithm-specific trainers initialize the building blocks for the algorithms from user-defined objectives. The interface between numerical optimization and algorithms enables the construction of customized trainers. For example, alternating the optimization through FM and DiffTRe trainers constitutes an approach to fuse top-down and bottom-up training [57].

We implemented the API following an object-oriented paradigm, defining the trainers as classes. The class-based implementations utilize inheritance to enforce consistency between different trainers and reuse functionalities for similar trainers. Illustrated in Fig. 2, the class hierarchy in `chemtrain` reflects the differences between single-point maximum-likelihood estimation and probabilistic learning, e.g., through providing access to a single optimal parameter set (`params`) or multiple samples (`list_of_params`). Likewise, the hierarchy reflects the respective dependence of DiffTRe and RM on the thermodynamic perturbation formalism, initializing reweighting procedures via the `add_statepoint` method. Additionally, using stateful trainers provides the advantage that the state of the optimizer, states of the algorithms, and training metrics do not have to be passed explicitly. In combination with consistency through inheritance, the access to states and metrics through common instance properties simplifies the usage of the algorithms. Moreover, this combination enables the implementation of more advanced trainers, such as the `InterleaveTrainer` trainer, combining training of FM and DiffTRe by synchronizing the model state (`params`) between successive optimization cycles.

Listing 2 illustrates the interplay between the modular structure at the lower level and the object-oriented API in the example of applying the DiffTRe algorithm to two different systems. The DiffTRe trainer accepts the functional definitions of objectives and optimizer and initializes the states of optimizer and algorithms. Due to the clear separation between data and computation, individual definitions, e.g., the `energy_fn_template`, are reusable for different standpoints or problem definitions. Since DiffTRe inherits from the `MLEtrainer` abstract base class like other MLE algorithms (see Fig. 2), training is possible through common methods such as `trainer.train(...)` and `trainer.save_energy_params(...)`.

```
# All statepoints share the same parameters
trainer = trainers.Difftre(init_params, optimizer)

# Multiple statepoints can be added, representing a specific
# system at a fixed thermodynamic state-point
trainer.add_statepoint(
    energy_fn_template, simulator_template, neighbor_fn, timings,
    state_kwargs, snapshot_fns, reference_state, targets,
    observable_fns)
```

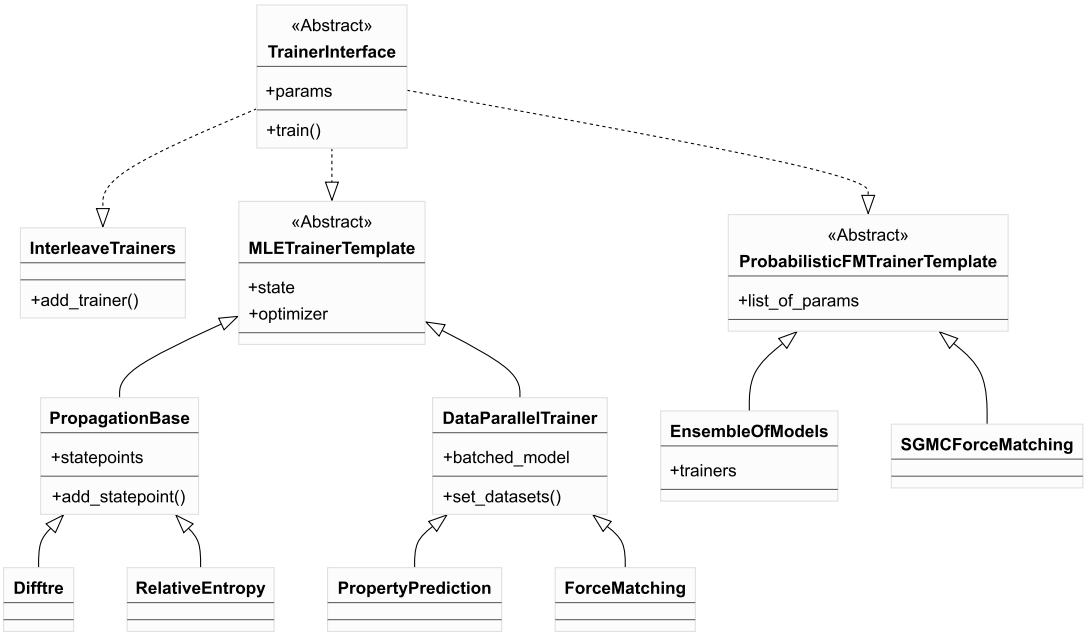


Fig. 2. Hierarchy of the trainer classes in `chemtrain`. The figure displays only a subset of all methods and attributes to illustrate similarities between the trainers.

Table 1
Overview of the most important building blocks of `chemtrain`.

Module	Content
`data`	**Data Loading:** Loading from `numpy` with shuffling/splitting/subsampling, initializing of JaxSGMC `DataLoaders` [70] **Data Processing:** Scaling to fractional coordinates, linear CG mapping from Multiscale CG method [32]
`ensemble`	**Reweighting:** Reference/Adaptive reweighting (NVT), BAR method [101,102] (NVT) **Simulation:** (Batched) execution of JAX, M.D. [65] simulations with equilibration/subsampling **Evaluation:** Batched snapshot computation **Utilities:** Initialization of simulator template
`quantity`	**Learnable Quantities:** dipole moment, partial charges [71] **Macrosopic Observables:** (linearized [103]) ensemble average, heat capacities [76,104], relative entropy **Utilities:** Helper functions to initialize DiffTRe targets
`learn`	**Algorithms:** DiffTRe [58], (probabilistic) Force Matching [28,69], Relative Entropy Minimization [28] **Utilities:** `pmap`/`shmap`-parallelization of loss/validation function, masked MSE/MAE functions
`trainers`	**Core Trainers:** DiffTRe [58], PropertyPrediction [71], ForceMatching [28], RelativeEntropy [28], SGMCForceMatching [69] **Advanced Trainers:** EnsembleOfModels, InterleaveTrainers

```
trainer.add_statepoint(
    energy_fn_template, ...
)

# Perform gradient-based training
trainer.train(num_updates)

# Save the trained energy parameters via pickle
trainer.save_energy_params('trainer.pkl', save_format='.pkl')

# Access training metrics, such as predictions, loss, ...
... = trainer.predictions
... = trainer.batch_losses
```

Listing 2: Initializing the DiffTRe algorithm.

4.2. Core modules

In the following section, we provide a closer description of the modules of `chemtrain`. Table 1 contains an overview of the most important building blocks arranged into the core modules of `chemtrain`.

4.2.1. `data` – loading and processing of reference data

`chemtrain` learns potential models via data-driven approaches from experimental or simulated reference data. In the bottom-up approach, the reference data consists of microscopic observations with typically higher spatial and temporal resolution than the macroscopic reference data used in top-down learning. Hence, efficient data management and stochastic optimization based on subsamples of the data are crucial to ensure the scalability of the bottom-up approaches.

In the `data.data_loaders` module, `chemtrain` provides an interface to JaxSGMC [70]. At its current state, JaxSGMC simplifies the usage of reference data in the `numpy` format, supporting multiple batching strategies and loading of data into `jit`-compiled functions. Moreover, following a modular concept like `chemtrain`, JaxSGMC enables setting up custom data-loading pipelines, e.g., to load reference data from other formats such as HDF5 [105]. Therefore, the `data.data_loaders` module ensures the extensibility of `chemtrain` to other data sources and data management strategies.

Additionally, the `data.preprocessing` module provides routines to load and pre-process reference data in the `numpy` format. These routines include typical tasks for data-driven learning, such as shuffling, subsampling, and data splitting. Moreover, the module provides preprocessing functionalities more specific to learning potential models, such as a CG mapping routine complying with the Multiscale CG method [32] (FM, section 3.2.1).

4.2.2. `ensemble` – simulating and importance sampling ensembles

The `ensemble` module complements the `data` module by providing utilities to generate data. In detail, the `ensemble` module divides into three submodules. First, the submodule `ensemble.sampling` provides routines to sample new states $\boldsymbol{r}$ from an ensemble $p_U(\boldsymbol{r})$, e.g., for DiffTRe and RM. Second, the submodule `ensemble.evaluation` contains functions to evaluate microscopic quantities $a(\boldsymbol{r})$ for large numbers of samples $\boldsymbol{r}$. Finally, the `ensemble.reweighting` module combines both functionalities to provide variants of thermodynamic perturbation theory.

In many cases, MD is a practical method to draw samples from an ensemble of molecular systems (section 2). Therefore, the `ensemble.sampling` module interfaces `chemtrain` to the MD integrators of JAX, M.D. [65] providing several classical integrators to sample from various ensembles, e.g., NVE, NVT, and NPT. The `ensemble.sampling` module combines these integrators with the flexible `energy_fn_template` (Listing 1) to obtain a parametric MD simulation routine. This routine follows a typical simulation procedure, discarding states during an equilibration cycle and keeping decorrelated states during the production cycle via subsampling. Moreover, the routine enables simulations at constant or time-dependent thermodynamic states, passing the properties of the current state to the energy function via keyword arguments (`**dynamic`, Listing 1). Additionally, through a fully functional implementation, the simulation routine can leverage JAX `vmap` to accelerate phase-space exploration by simulating multiple systems in parallel. In principle, the interface also supports advanced simulation methods such as tempered/meta-dynamics replica exchange [106,107].

A crucial part of top-down and bottom-up training algorithms is the computation of microscopic quantities $a(\boldsymbol{r})$ for many sampled conformations $\boldsymbol{r}$. The submodule `ensemble.evaluation` provides functions to perform this computation efficiently by performing automatically batched computations via `vmap`. In line with the modular concept of `chemtrain`, the implementation of the microscopic quantities follows a simple but flexible interface, displayed in Listing 3. Derived from the states of the JAX, M.D. simulators, the `state` argument bundles the internal state of a system. Essentially, this state contains the particle positions $\boldsymbol{r}$ but can also include additional properties, such as the box size for NPT ensembles. The keyword arguments provide external parameters, e.g., the thermostat pressure, temperature, and learnable parameters θ. Moreover, the keyword arguments can provide pre-extracted features, such as the neighbor list, shared among the microscopic quantities for computational efficiency. Additional pre-extracted features are simple to specify, following the same protocol as the microscopic quantities.

```
def compute_fn(state, neighbor=None, **kwargs) -> PyTree:
    # Computation, e.g., RDF or pressure
    ...
    return quantity_snapshot
```

Listing 3: Atomistic (snapshot) quantity

The submodule `ensemble.reweighting` implements different variants of thermodynamic perturbation theory [81]. Combined with the `sampling` and `evaluation` submodules, one variant is a streamlined version of umbrella sampling [82] for NVT and NPT ensembles with automatic re-sampling of reference states $\boldsymbol{r}$. Crucial for the DiffTRe and RM algorithms, this umbrella sampling variant provides an efficient end-to-end differentiable routine to compute macroscopic equilibrium properties (section 3.1). Moreover, by employing the BAR algorithm [101,102] and Zwanzig's relation [81] between successive resampling steps, this umbrella sampling variant estimates the change of free energy ΔQ and entropy ΔS between the initial and current potential U_θ on-the-fly [55]. Additional routines enable reweighting based on pre-computed data and between different thermodynamic state points.

4.2.3. `quantity` – microscopic and macroscopic observables

The DiffTRe algorithm provides a method to learn macroscopic equilibrium properties $\mathcal{A}$ (section 3.1). Therefore, the `quantity.observables` module contains implementations of $\mathcal{A}$ from microscopic properties $a(\boldsymbol{r})$ compatible with the underlying thermodynamic perturbation theory. In the simplest scenario, the macroscopic properties are weighted ensemble averages of a single type of microscopic properties, e.g., the instantaneous radial distribution function or microscopic pressure. However, some thermodynamic quantities, such as the isobaric heat capacity [104], depend on multiple ensemble averages formed by multiple microscopic properties. To flexibly support such more complicated observables, `chemtrain` passes all microscopic quantities and the current weights to the compute functions of the macroscopic observables. Listing 4 provides an example implementation of an observable, computing the variance $\mathrm{Var}_U[a] = \langle a^2 \rangle_U - \langle a \rangle_U^2$ of an instantaneous observable $a(\boldsymbol{r})$. Due to passing all computed microscopic predictions, the logic of selecting the required quantities remains within the compute function for the macroscopic observable, and microscopic predictions are only computed once. Moreover, the computation of more involved observables, e.g., containing the variance of a microscopic property, is possible. Nevertheless, the separation between microscopic and macroscopic observables can complicate the usage. Therefore, the additional `quantity.targets` module aims to mitigate this complication by providing routines to set up macroscopic observables alongside required microscopic observables for common quantities, such as the radial and angular distribution functions or the pressure.

```
def obs_variance(snapshots, weights=None, **statepoint_kwargs):
    # A dynamic definition of the statpoint is possible
    del statepoint_kwargs # Unused by computation

    # In general, the compute function has access to all
    # snapshots
    snapshot = snapshots['quantity_key']
    snapshot_sq = snapshot ** 2

    # Weights are only necessary for the DiffTRe algorithm
    if weights is not None:
        weights *= weights.size
        snapshot = (snapshot.T * weights).T
        snapshot_sq = (snapshot_sq.T * weights).T

    # Variance is Var[x] = E[xx] - E[x]E[x]
    var = jnp.mean(snapshot_sq, axis=0)
    var -= jnp.mean(snapshot, axis=0) ** 2

return var
```

Listing 4: Example implementation of the weighted ensemble average.

In addition to modeling complicated potential energy surfaces, NNs can also accurately model other molecular or atomistic properties, such as partial charges. Therefore, the `quantity.property_prediction` module extends the classical microscopic quantities, e.g., correlation functions or pressures, with learnable quantities such as partial charges [71]. In detail, the submodule provides tools to derive property predictions from NNs, interfacing these predictors with the `compute_fn` protocol (Listing 3) expected by the `ensemble.evaluation` module.

4.2.4. `learn` — (deep) learning algorithms

The `learn` module contributes building blocks for standard training algorithms. These building blocks comprise general utilities and concrete routines for the core algorithms of `chemtrain`. For example, the module provides general utilities that evaluate the model, compute the loss, and perform the gradient update on multiple devices using JAX `pmap` and `shmap` transformations. Trainers can apply such general utilities or other JAX transformations to functional implementations of larger algorithm parts, i.e., concrete routines, to efficiently employ algorithms. Therefore, the `data` module contains the highest functional layer of the FM, RM, and DiffTRe algorithms.

4.3. `trainers` — high-level API to algorithms

The `trainers` module contains the high level API of `chemtrain`. Besides the wrappers around the core algorithms described in sec-

tion 4.1, this API enables the creation of more advanced training routines. For example, Listing 5 demonstrates the fusion of the FM and DiffTRe algorithm through the higher-level `InterleaveTrainers` trainer. Additionally, the `trainers.extensions` submodule provides plugin-functionalities to extend routines of the trainers, e.g., to enhance their usability. For example, the submodule provides a decorator to the DiffTRe and FM trainers to log the training statistics to the ML framework Weights & Biases [108] after each epoch. Therefore, the trainers API provides much flexibility to extend the functionality and usability of the core training algorithms without the need to make difficult changes to the core of the `chemtrain` library.

```
# Allows combining the trainers, executing them in the
# order in which they were added
trainer = trainers.InterleaveTrainers(full_checkpoint=False)

# Run force matching 10 epochs before running difftre for
# 2 epochs
trainer.add_trainer(fm_trainer, num_updates=10,
    name='Force Matching')
trainer.add_trainer(difftre_trainer, num_updates=2,
    name='DiffTRe')

# Training as usual for MLE trainers
trainer.train(100, checkpoint_frequency=10)
```

Listing 5: Combining multiple trainers.

5. Examples

In the following section, we illustrate the usage of `chemtrain` in two examples. In the first example, we combine the bottom-up approaches FM and RM to efficiently learn an accurate structural representation of a heavy-atom alanine dipeptide model in implicit water [28]. In the second example, we combine bottom-up and top-down learning via the FM and DiffTRe algorithms to train an atomistic titanium model consistent with experimental measurements of thermodynamic properties [57]. The examples briefly demonstrate the importance of combining multiple learning strategies to efficiently develop accurate potential models. We refer to the original papers and the reference documentation for detailed descriptions and results.

5.1. Bottom-up CG model of alanine dipeptide in implicit water

Alanine Dipeptide is a classical benchmark system for CG training methods to test their ability to model free energy surfaces with multiple meta-stable states. The reference data and training setup follow the original paper [28]. In short, the implicit solvent CG model consists of the 10 heavy atoms of Alanine Dipeptide, without hydrogen atoms or water molecules. The reference data was derived from a 100 ns atomistic reference trajectory at $T_{\text{ref}} = 300$ K. Building on the Δ-learning ansatz [99], the potential model consists of a learnable correction $\Delta U(\theta, \mathbf{R})$ via the DimeNet++ [19,58] message-passing neural network and a prior potential

$$U^{\text{Prior}}(\mathbf{R}) = \sum_i U_i^H(b_i) + \sum_j U_j^H(\alpha_j) + \sum_k U_k^D(\omega_k) \tag{24}$$

based on harmonic terms U^H for all bonds b_i and angles α_j and cosine series potentials U_i^D for the proper dihedral angles of the backbone ω_i.

Listing 6 illustrates the training setup in `chemtrain`. First, we set up the prior potential by defining the parameters in a `toml` file and generating the topology from the initial structure. After defining the complete potential following Listing 1, we train the model via FM for 100 epochs. Finally, we set up a Langevin thermostat to sample from the learned ensemble and fine-tune the model by RM for 50 updates.

```
# Define prior potential
force_field = prior.ForceField.load_ff(
    "../_data/alanine_heavy.toml")
top = mdtraj.load_topology("../_data/alanine_heavy_2_7nm.gro")
topology = prior.Topology.from_mdtraj(top, mapping)

# setup total model
def energy_fn_template(energy_params): ...

# Set up and train with force matching
force_matching = trainers.ForceMatching(
    init_params, fm_optimizer, energy_fn_template, nbrs_init,
    batch_per_device=500 // len(jax.devices())
)
force_matching.set_datasets({
    'R': position_dataset[:5e5, ...],
    'F': force_dataset[:5e5, ...]
    }, train_ratio=0.7
)
force_matching.train(100)
force_matching.save_energy_params(
    "alanine_dipeptide_fm_params.pkl", '.pkl')
force_matching.save_trainer(
    "alanine_dipeptide_fm_trainer.pkl", '.pkl')

# Define simulator, RM trainer, add simulation and fine-tune
# model
init_ref_state, sim_template = \
    sampling.initialize_simulator_template(
    simulate.nvt_langevin, shift_fn=shift_fn, nbrs=nbrs_init,
    extra_simulator_kwargs={"kT": 300. * quantity.kb,
    "gamma": 100, "dt": 0.002}
)
rm_post_fm = trainers.RelativeEntropy(
    force_matching_params, optimizer, reweight_ratio=1.1,
    energy_fn_template=energy_fn_template)
rm_post_fm.add_statepoint(
    position_dataset[:4e5, ...],
    energy_fn_template, sim_template, neighbor_fn,
    re_timings, state_kwargs, reference_state,
    reference_batch_size=4e5,
    vmap_batch=100, resample_simstates=True)
relative_entropy.train(50)
```

Listing 6: Training a Δ-learning model following Listing 1 for Alanine Dipeptide with force matching and relative entropy minimization.

After training, we compare the FM model to an RM model trained for 300 updates and the FM model fine-tuned via 50 RM updates. Therefore, we sample 100 ns worth of MD simulations for each model and compare the obtained dihedral angle distribution to the atomistic reference (Fig. 3). The FM model deviates more strongly from the reference density than the RM model. This performance difference to the RM model can arise from the finite model capacity that affects the optimal FM solution [109] and errors from the MD simulation [28]. Moreover, RM converges evidently to its optimal potential with substantially fewer data [28,110]. However, when retraining the FM model via 50 RM updates, the model matches the atomistic reference as accurately as trained solely via RM for 300 updates. We attribute the improved performance of the hybrid training largely to the improved initial model state provided by FM. As the targets of FM and RM partially align [28], performing RM from the FM model rather than a random initialization circumvents traversing a larger distance in parameter space. Therefore, RM can directly fine-tune a more suitable initial model, reducing computational demands.

5.2. Atomistic model of titanium via experimental and simulation data fusion

This example illustrates the importance of fusing experimental and simulation reference data to obtain an atomistic model of titanium consistent with ab initio computations and experimental measurements. The training procedure and reference data follow the paper by Röcken and Zavadlav [57]. From the original paper, we take the DimeNet++ model pre-trained on a curated dataset of energies, forces, and virials computed via density functional theory for various conformations of bulk titanium [111]. Additionally, we take the experimental measurements of bulk elastic constants at multiple temperatures [112].

Illustrated in Listing 7, we employ FM on the ab initio data and DiffTRe on the experimental data at 323 and 923 K to fuse microscopic and macroscopic reference data in `chemtrain`. We first set up both

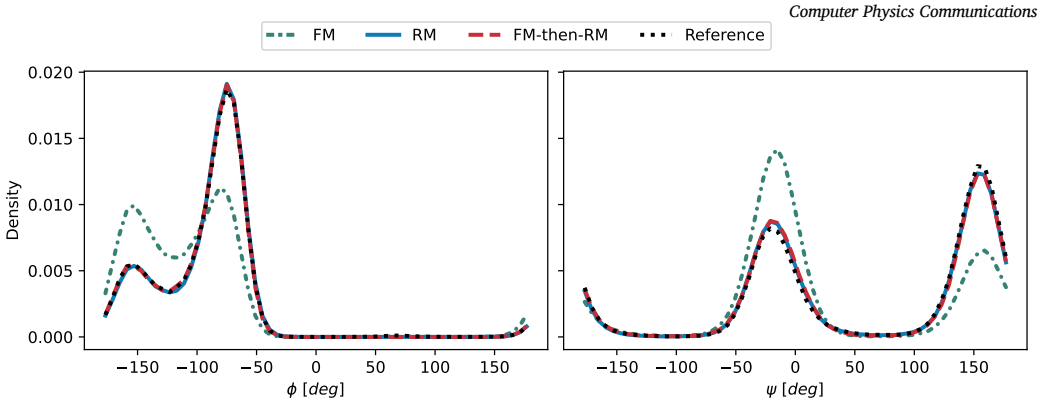


Fig. 3. Density histograms of the backbone dihedral angels of alanine dipeptide from simulations with Force Matching (FM), Relative Entropy Minimization (RM), and FM-then-RM trained models in comparison to an atomistic reference simulation.

trainers separately, defining the computations of microscopic quantities for FM and DiffTRe and the computations of macroscopic observables only for DiffTRe. Then, we leverage the `InterleaveTrainers` trainer for 100 repetitions, each sequentially training the model for one epoch of FM and one update for all statepoints of DiffTRe.

```
# Initialize an FM trainer with pre-trained params
trainer_fm = trainers.ForceMatching(
    pretrained_params, optimizer_fm, energy_fn_template,
    nbrs_init, batch_per_device=30, batch_cache=8,
    gammas={'W': 4e-6,'U': 1e-6, 'F': 1e-2},
    additional_targets={'W': virial_snapshot_fn},
    weights_keys={'W': 'W_weights'}
)
# Load training data analogous to the previous example
trainer_fm.set_datasets(...)

# Define the observables for top-down training, the
# required snapshots, and the simulator to sample states
observables = {
    'C_const': observables.init_born_stiffness_tensor(...),
    'pressure': observables.init_traj_mean_fn('pressure')
}
compute_fns = {
    'born_stiffness': born_stiffness_snapshot_fn,
    'born_stress': born_stress_snapshot_fn,
    'pressure': pressure_snapshot_fn
}
# Analogous to previous example
init_ref_state, sim_template = sampling.
     initialize_simulator_template(...)

# Initialize DiffTRe to train elastic constants and pressure
trainer_difftre = trainers.Difftre(
    init_params, optimizer_difftre, sim_batch_size=-1,
    reweight_ratio=1.0
)

# Add a separate statepoint for each temperature. We can re-use
# the snapshot and observable functions for all statepoints
for temp in [323, 923]:
    targets = {
        'C_const': {'target': exp_data[temp], 'gamma': 1e-10},
        'pressure': {'target': 1. / 16.6054, 'gamma': 1e-9}}
    # Analogous to previous example
    reference_state = init_ref_state(...)

    trainer_difftre.add_statepoint(
        energy_fn_template, sim_template, neighbor_fn_difftre,
        timings, state_kwargs=state_kwargs[temp],
        reference_state=reference_state, targets=targets,
        observables=observables, quantities=compute_fns)

# Optimize sequentially on both objectives
trainer_fused = trainers.InterleaveTrainers(sequential=True)
trainer_fused.add_trainer(trainer_fm, num_updates=1, name='FM')
trainer_fused.add_trainer(trainer_difftre, num_updates=1,
    name='DiffTRe')
trainer_fused.train(100)
```

Listing 7: Fused bottom-up and top-down learning of an atomistic NN model of titanium.

To compare both models, we compute the elastic constants in a 70 ps long reference simulation withholding 10 ps of equilibration. Fig. 4 shows that the fusion of top-down and bottom-up learning can improve the agreement of the model and experiments. We expect further improvement by training for more epochs and using a longer sampling interval than 20 ps for training with DiffTRe.

6. Conclusion

In this work, we presented `chemtrain`, a framework that enables the training of sophisticated NN potentials through customizable training routines. Similarities between commonly applied top-down and bottom-up training procedures allow the decomposition of the algorithms into standard building blocks. Therefore, we implemented `chemtrain` in a modular structure, following a functional paradigm at the lower level to leverage JAX transformations to obtain scalable and reusable building blocks. An object-oriented API built on the modular structure enables a simple combination or extension of algorithms.

Our experiments demonstrate that combining algorithms as performed for classical force fields is also beneficial for training NN potential models. On the one hand, combining methods enables a pretraining of the potential with less expensive methods to reduce the cost of more computationally demanding but more accurate or data-efficient methods. On the other hand, supplementing bottom-up methods with top-down approaches expands the amount of usable data and enforces predictions consistent with experiments. Therefore, we expect that `chemtrain` can contribute to expediting the success of NN potential models for molecular simulations.

For the future, we plan to extend `chemtrain` by providing further advanced training routines. For example, active learning can improve data efficiency [23] and is, in principle, straightforward to add due to the flexible API of `chemtrain`. Additionally, we plan to extend the supported macroscopic observables, such as dynamical observables and better free energy perturbation estimates, by generalizing the DiffTRe algorithm. Moreover, we aim to accelerate the DiffTRe algorithm by integrating learnable advanced sampling schemes with `chemtrain`. As it is unlikely that JAX, M.D. will catch up with established MD packages regarding robustness, flexibility, and computational efficiency in the foreseeable future, we also intend to tighten the integration of `chemtrain` with classical MD software [7–10] to simplify the deployment of the trained NN potentials to large scale simulations.

CRediT authorship contribution statement

Paul Fuchs: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Stephan Thaler:** Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization. **Sebastien Röcken:** Writing – original draft, Visualization, Validation, Software, Methodology,

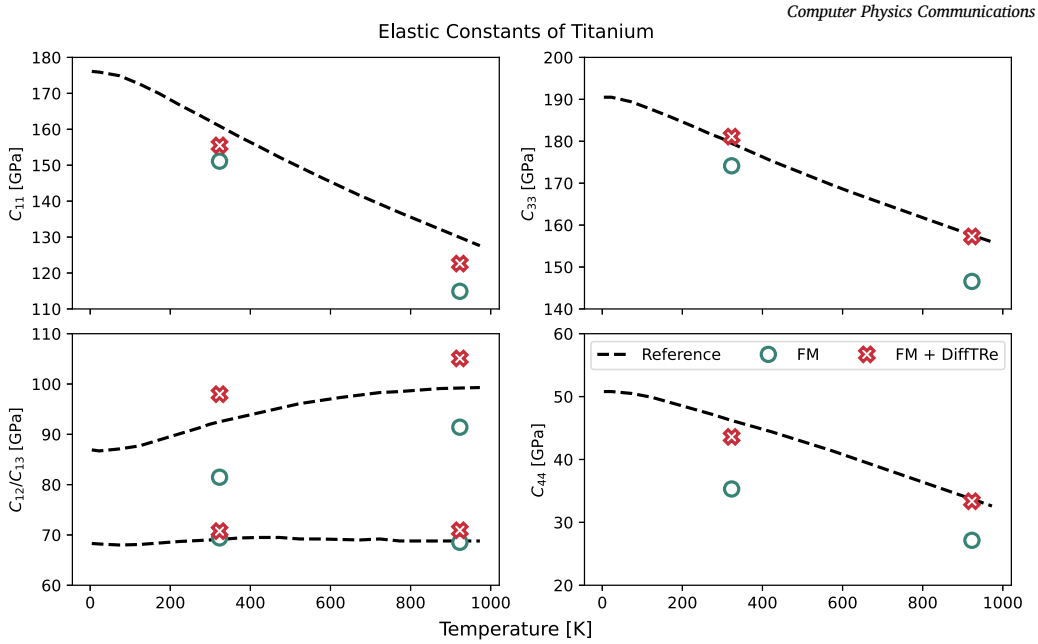


Fig. 4. Predicted elastic constants. The dashed line denotes the experimental reference, the circles denote the prediction of the bottom-up trained model via Force Matching (FM), and the stars denote the predictions after sequentially training bottom-up with FM and top-down with Differential Trajectory Reweighting (DiffTRe).

Investigation, Formal analysis. **Julija Zavadlav:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Julija Zavadlav reports financial support was provided by European Research Council. Julija Zavadlav reports financial support was provided by German Research Foundation. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was funded by ERC StG 101077842 – SupraModel and Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 534045056.

Data availability

The data to reproduce the results is publicly available and can be obtained through the open-sourced code in the GitHub repository https://github.com/tummfm/chemtrain.

References

[1] J. Hartmann, M. Zacharias, Mechanism of collagen folding propagation studied by Molecular Dynamics simulations, PLoS Comput. Biol. 17 (6) (2021) e1009079, https://doi.org/10.1371/journal.pcbi.1009079.

[2] C. Li, A. Strachan, Molecular dynamics predictions of thermal and mechanical properties of thermoset polymer EPON862/DETDA, Polymer 52 (13) (2011) 2920–2928, https://doi.org/10.1016/j.polymer.2011.04.041.

[3] J.F. Fatriansyah, D. Dhaneswara, M.H. Abdurrahman, F.R. Kuskendrianto, M.B. Yusuf, Molecular dynamics simulation of hydrogen adsorption on Silica, IOP Conf. Ser., Mater. Sci. Eng. 478 (1) (2019) 012034, https://doi.org/10.1088/1757-899X/478/1/012034.

[4] W.D. Cornell, P. Cieplak, C.I. Bayly, I.R. Gould, K.M. Merz, D.M. Ferguson, D.C. Spellmeyer, T. Fox, J.W. Caldwell, P.A. Kollman, A second generation force field for the simulation of proteins, nucleic acids, and organic molecules, J. Am. Chem. Soc. 117 (19) (1995) 5179–5197, https://doi.org/10.1021/ja00124a002.

[5] A.M. Nikitin, Y.V. Milchevskiy, A.P. Lyubartsev, A new AMBER-compatible force field parameter set for alkanes, J. Mol. Model. 20 (3) (2014) 2143, https://doi.org/10.1007/s00894-014-2143-6.

[6] S.J. Marrink, A.H. de Vries, A.E. Mark, Coarse grained model for semiquantitative lipid simulations, J. Phys. Chem. B 108 (2) (2004) 750–760, https://doi.org/10.1021/jp036508g.

[7] A.P. Thompson, H.M. Aktulga, R. Berger, D.S. Bolintineanu, W.M. Brown, P.S. Crozier, P.J. in 't Veld, A. Kohlmeyer, S.G. Moore, T.D. Nguyen, R. Shan, M.J. Stevens, J. Tranchida, C. Trott, S.J. Plimpton, LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Comput. Phys. Commun. 271 (2022) 108171, https://doi.org/10.1016/j.cpc.2021.108171.

[8] M.J. Abraham, T. Murtola, R. Schulz, S. Páll, J.C. Smith, B. Hess, E. Lindahl, GROMACS: high performance molecular simulations through multi-level parallelism from laptops to supercomputers, SoftwareX 1–2 (2015) 19–25, https://doi.org/10.1016/j.softx.2015.06.001.

[9] J.A. Anderson, C.D. Lorenz, A. Travesset, General purpose molecular dynamics simulations fully implemented on graphics processing units, J. Comput. Phys. 227 (10) (2008) 5342–5359, https://doi.org/10.1016/j.jcp.2008.01.047.

[10] J.A. Anderson, J. Glaser, S.C. Glotzer, HOOMD-blue: a Python package for high-performance molecular dynamics and hard particle Monte Carlo simulations, Comput. Mater. Sci. 173 (2020) 109363, https://doi.org/10.1016/j.commatsci.2019.109363.

[11] V. Tozzini, Multiscale modeling of proteins, Acc. Chem. Res. 43 (2) (2010) 220–230, https://doi.org/10.1021/ar9001476.

[12] J. Zavadlav, M.N. Melo, S.J. Marrink, M. Praprotnik, Adaptive resolution simulation of an atomistic protein in MARTINI water, J. Chem. Phys. 140 (5) (2014) 054114, https://doi.org/10.1063/1.4863329.

[13] J. Zavadlav, R. Podgornik, M. Praprotnik, Order and interactions in DNA arrays: multiscale molecular dynamics simulation, Sci. Rep. 7 (1) (2017) 4775, https://doi.org/10.1038/s41598-017-05109-2.

[14] S. Thaler, M. Praprotnik, J. Zavadlav, Back-mapping augmented adaptive resolution simulation, J. Chem. Phys. 153 (16) (2020) 164118, https://doi.org/10.1063/5.0025728.

[15] J. Behler, Constructing high-dimensional neural network potentials: a tutorial review, Int. J. Quant. Chem. 115 (16) (2015) 1032–1050, https://doi.org/10.1002/qua.24890.

[16] J.S. Smith, O. Isayev, A.E. Roitberg, ANI-1: an extensible neural network potential with DFT accuracy at force field computational cost, Chem. Sci. 8 (4) (2017) 3192–3203, https://doi.org/10.1039/C6SC05720A.

[17] K.T. Schütt, P.-J. Kindermans, H.E. Sauceda, S. Chmiela, A. Tkatchenko, K.-R. Müller, SchNet: a continuous-filter convolutional neural network for modeling quantum interactions, arXiv:1706.08566, https://doi.org/10.48550/arXiv.1706.08566, Dec. 2017.

[18] J. Gasteiger, J. Groß, S. Günnemann, Directional message passing for molecular graphs, arXiv:2003.03123, https://doi.org/10.48550/arXiv.2003.03123, Apr. 2022.

[19] J. Gasteiger, S. Giri, J.T. Margraf, S. Günnemann, Fast and uncertainty-aware directional message passing for non-equilibrium molecules, arXiv:2011.14115, https://doi.org/10.48550/arXiv.2011.14115, Apr. 2022.
[20] S. Batzner, A. Musaelian, L. Sun, M. Geiger, J.P. Mailoa, M. Kornbluth, N. Molinari, T.E. Smidt, B. Kozinsky, E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials, Nat. Commun. 13 (1) (2022) 1–11, https://doi.org/10.1038/s41467-022-29939-5.
[21] A. Musaelian, S. Batzner, A. Johansson, L. Sun, C.J. Owen, M. Kornbluth, B. Kozinsky, Learning local equivariant representations for large-scale atomistic dynamics, Nat. Commun. 14 (1) (2023) 579, https://doi.org/10.1038/s41467-023-36329-y.
[22] J. Behler, Atom-centered symmetry functions for constructing high-dimensional neural network potentials, J. Chem. Phys. 134 (7) (2011) 074106, https://doi.org/10.1063/1.3553717.
[23] J.S. Smith, B.T. Nebgen, R. Zubatyuk, N. Lubbers, C. Devereux, K. Barros, S. Tretiak, O. Isayev, A.E. Roitberg, Approaching coupled cluster accuracy with a general-purpose neural network potential through transfer learning, Nat. Commun. 10 (1) (2019) 2903, https://doi.org/10.1038/s41467-019-10827-4.
[24] R. Iftimie, P. Minary, M.E. Tuckerman, Ab initio molecular dynamics: concepts, recent developments, and future trends, Proc. Natl. Acad. Sci. 102 (19) (2005) 6654–6659, https://doi.org/10.1073/pnas.0500193102.
[25] C.E. Tzeliou, M.A. Mermigki, D. Tzeli, Review on the QM/MM methodologies and their application to metalloproteins, Molecules 27 (9) (2022) 2660, https://doi.org/10.3390/molecules27092660.
[26] I. Zaporozhets, C. Clementi, Multibody terms in protein coarse-grained models: a top-down perspective, J. Phys. Chem. B 127 (31) (2023) 6920–6927, https://doi.org/10.1021/acs.jpcb.3c04493.
[27] Y. Chen, A. Krämer, N.E. Charron, B.E. Husic, C. Clementi, F. Noé, Machine learning implicit solvation for molecular dynamics, J. Chem. Phys. 155 (8) (2021) 084101, https://doi.org/10.1063/5.0059915.
[28] S. Thaler, M. Stupp, J. Zavadlav, Deep coarse-grained potentials via relative entropy minimization, J. Chem. Phys. 157 (24) (2022) 244103, https://doi.org/10.1063/5.0124538.
[29] W.G. Noid, Perspective: coarse-grained models for biomolecular systems, J. Chem. Phys. 139 (9) (2013) 090901, https://doi.org/10.1063/1.4818908.
[30] M.S. Shell, The relative entropy is fundamental to multiscale and inverse thermodynamic problems, J. Chem. Phys. 129 (14) (2008) 144108, https://doi.org/10.1063/1.2992060.
[31] F. Ercolessi, J.B. Adams, Interatomic potentials from first-principles calculations: the force-matching method, Europhys. Lett. 26 (8) (1994) 583–588, https://doi.org/10.1209/0295-5075/26/8/005.
[32] W.G. Noid, J.-W. Chu, G.S. Ayton, V. Krishna, S. Izvekov, G.A. Voth, A. Das, H.C. Andersen, The multiscale coarse-graining method. I. A rigorous bridge between atomistic and coarse-grained models, J. Chem. Phys. 128 (24) (2008) 244114, https://doi.org/10.1063/1.2938860.
[33] A.P. Lyubartsev, A. Laaksonen, Calculation of effective interaction potentials from radial distribution functions: a reverse Monte Carlo approach, Phys. Rev. E 52 (4) (1995) 3730–3737, https://doi.org/10.1103/PhysRevE.52.3730.
[34] D. Reith, M. Pütz, F. Müller-Plathe, Deriving effective mesoscale potentials from atomistic simulations, J. Comput. Chem. 24 (13) (2003) 1624–1636, https://doi.org/10.1002/jcc.10307.
[35] A.B. Norgaard, J. Ferkinghoff-Borg, K. Lindorff-Larsen, Experimental parameterization of an energy function for the simulation of unfolded proteins, Biophys. J. 94 (1) (2008) 182–192, https://doi.org/10.1529/biophysj.107.108241.
[36] A.D. White, G.A. Voth, Efficient and minimal method to bias molecular simulations with experimental data, J. Chem. Theory Comput. 10 (8) (2014) 3023–3030, https://doi.org/10.1021/ct500320c.
[37] A.D. White, J.F. Dama, G.A. Voth, Designing free energy surfaces that match experimental data with metadynamics, J. Chem. Theory Comput. 11 (6) (2015) 2451–2460, https://doi.org/10.1021/acs.jctc.5b00178.
[38] M. Rupp, E. Küçükbenli, G. Csányi, Guest editorial: special topic on software for atomistic machine learning, J. Chem. Phys. 161 (6) (2024) 060401, https://doi.org/10.1063/5.0228461.
[39] H. Wang, L. Zhang, J. Han, W. E, DeePMD-kit: a deep learning package for many-body potential energy representation and molecular dynamics, Comput. Phys. Commun. 228 (2018) 178–184, https://doi.org/10.1016/j.cpc.2018.03.016.
[40] J. Zeng, D. Zhang, D. Lu, P. Mo, Z. Li, Y. Chen, M. Rynik, L. Huang, Z. Li, S. Shi, Y. Wang, H. Ye, P. Tuo, J. Yang, Y. Ding, Y. Li, D. Tisi, Q. Zeng, H. Bao, Y. Xia, J. Huang, K. Muraoka, Y. Wang, J. Chang, F. Yuan, S.L. Bore, C. Cai, Y. Lin, B. Wang, J. Xu, J.-X. Zhu, C. Luo, Y. Zhang, R.E.A. Goodall, W. Liang, A.K. Singh, S. Yao, J. Zhang, R. Wentzcovitch, J. Han, J. Liu, W. Jia, D.M. York, W. E, R. Car, L. Zhang, H. Wang, DeePMD-kit v2: a software package for deep potential models, J. Chem. Phys. 159 (5) (2023) 054801, https://doi.org/10.1063/5.0155600.
[41] K.T. Schütt, P. Kessel, M. Gastegger, K.A. Nicoli, A. Tkatchenko, K.-R. Müller, SchNetPack: a deep learning toolbox for atomistic systems, J. Chem. Theory Comput. 15 (1) (2019) 448–455, https://doi.org/10.1021/acs.jctc.8b00908.
[42] K.T. Schütt, S.S.P. Hessmann, N.W.A. Gebauer, J. Lederer, M. Gastegger, SchNetPack 2.0: a neural network toolbox for atomistic machine learning, J. Chem. Phys. 158 (14) (2023) 144801, https://doi.org/10.1063/5.0138367.
[43] R. Barrett, M. Chakraborty, D. Amirkulova, H. Gandhi, A. White, A GPU-accelerated machine learning framework for molecular simulation: HOOMD-blue with TensorFlow, https://doi.org/10.26434/chemrxiv.8019527.v3, 2019.
[44] A. Khorshidi, A.A. Peterson, Amp: a modular approach to machine learning in atomistic simulations, Comput. Phys. Commun. 207 (2016) 310–324, https://doi.org/10.1016/j.cpc.2016.05.010.
[45] M. Shuaibi, Y. Hu, X. Lei, B.M. Comer, M. Adams, J. Paras, R.Q. Chen, E. Musa, J. Musielewicz, A.A. Peterson, A.J. Medford, Z. Ulissi, AmpTorch: a Python package for scalable fingerprint-based neural network training on multi-element systems with integrated uncertainty quantification, Zenodo, https://doi.org/10.5281/zenodo.8151492, Jul. 2023.
[46] T. Plé, O. Adjoua, L. Lagardère, J.-P. Piquemal, FeNNol: an efficient and flexible library for building force-field-enhanced neural network potentials, arXiv:2405.01491, https://doi.org/10.48550/arXiv.2405.01491, May 2024.
[47] N. Artrith, A. Urban, An implementation of artificial neural-network potentials for atomistic materials simulations: performance for TiO2, Comput. Mater. Sci. 114 (2016) 135–150, https://doi.org/10.1016/j.commatsci.2015.11.047.
[48] A.M. Cooper, J. Kästner, A. Urban, N. Artrith, Efficient training of ANN potentials by including atomic forces via Taylor expansion and application to water and a transition-metal oxide, npj Comput. Mater. 6 (1) (2020) 1–14, https://doi.org/10.1038/s41524-020-0323-8.
[49] A. Singraber, J. Behler, C. Dellago, Library-based LAMMPS implementation of high-dimensional neural network potentials, J. Chem. Theory Comput. 15 (3) (2019) 1827–1840, https://doi.org/10.1021/acs.jctc.8b00770.
[50] B. Kolb, L.C. Lentz, A.M. Kolpak, Discovering charge density functionals and structure-property relationships with PROPhet: a general framework for coupling machine learning and first-principles methods, Sci. Rep. 7 (1) (2017) 1192, https://doi.org/10.1038/s41598-017-01251-z.
[51] B. Ramsundar, P. Eastman, P. Walters, V. Pande, K. Leswing, Z. Wu, Deep learning for the life sciences, O'Reilly media, 2019, https://www.amazon.com/Deep-Learning-Life-Sciences-Microscopy/dp/1492039837.
[52] Z. Fan, Y. Wang, P. Ying, K. Song, J. Wang, Y. Wang, Z. Zeng, K. Xu, E. Lindgren, J.M. Rahm, A.J. Gabourie, J. Liu, H. Dong, J. Wu, Y. Chen, Z. Zhong, J. Sun, P. Erhart, Y. Su, T. Ala-Nissila, GPUMD: a package for constructing accurate machine-learned potentials and performing highly efficient atomistic simulations, J. Chem. Phys. 157 (11) (2022) 114801, https://doi.org/10.1063/5.0106617.
[53] K. Song, S. Käser, K. Töpfer, L.I. Vazquez-Salazar, M. Meuwly, PhysNet meets CHARMM: a framework for routine machine learning/molecular mechanics simulations, J. Chem. Phys. 159 (2) (2023) 024125, https://doi.org/10.1063/5.0155992.
[54] Y.-P. Huang, Y. Xia, L. Yang, J. Wei, Y.I. Yang, Y.Q. Gao, SPONGE: a GPU-accelerated molecular dynamics package with enhanced sampling and AI-driven algorithms, Chin. J. Chem. 40 (1) (2022) 160–168, https://doi.org/10.1002/cjoc.202100456.
[55] S. Röcken, A.F. Burnet, J. Zavadlav, Predicting solvation free energies with an implicit solvent machine learning potential, arXiv:2406.00183, May 2024.
[56] T. Fröhlking, M. Bernetti, N. Calonaci, G. Bussi, Toward empirical force fields that match experimental observables, J. Chem. Phys. 152 (23) (2020) 230902, https://doi.org/10.1063/5.0011346.
[57] S. Röcken, J. Zavadlav, Accurate machine learning force fields via experimental and simulation data fusion, npj Comput. Mater. 10 (1) (2024) 1–10, https://doi.org/10.1038/s41524-024-01251-4.
[58] S. Thaler, J. Zavadlav, Learning neural network potentials from experimental data via Differentiable Trajectory Reweighting, Nat. Commun. 12 (1) (2021) 6884, https://doi.org/10.1038/s41467-021-27241-4.
[59] C. Navarro, M. Majewski, G. De Fabritiis, Top-down machine learning of coarse-grained protein force fields, J. Chem. Theory Comput. 19 (21) (2023) 7518–7526, https://doi.org/10.1021/acs.jctc.3c00638.
[60] D.-W. Li, R. Brüschweiler, Iterative optimization of molecular mechanics force fields from NMR data of full-length proteins, J. Chem. Theory Comput. 7 (6) (2011) 1773–1782, https://doi.org/10.1021/ct200094b.
[61] S.P. Carmichael, M.S. Shell, A new multiscale algorithm and its application to coarse-grained peptide models for self-assembly, J. Phys. Chem. B 116 (29) (2012) 8383–8393, https://doi.org/10.1021/jp2114994.
[62] L.-P. Wang, J. Chen, T. Van Voorhis, Systematic parametrization of polarizable force fields from quantum chemistry data, J. Chem. Theory Comput. 9 (1) (2013) 452–460, https://doi.org/10.1021/ct300826t.
[63] X. Wang, J. Li, L. Yang, F. Chen, Y. Wang, J. Chang, J. Chen, W. Feng, L. Zhang, K. Yu, DMFF: an open-source automatic differentiable platform for molecular force field development and molecular dynamics simulation, J. Chem. Theory Comput. 19 (17) (2023) 5897–5909, https://doi.org/10.1021/acs.jctc.2c01297.
[64] S. Doerr, M. Majewski, A. Pérez, A. Krämer, C. Clementi, F. Noe, T. Giorgino, G. De, Fabritiis, TorchMD: a deep learning framework for molecular simulations, J. Chem. Theory Comput. 17 (4) (2021) 2355–2363, https://doi.org/10.1021/acs.jctc.0c01343.
[65] S.S. Schoenholz, E.D. Cubuk, JAX, M.D. A framework for differentiable physics, J. Stat. Mech. Theory Exp. 2021 (12) (2021) 124016, https://doi.org/10.1088/1742-5468/ac3ae9.
[66] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G.S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mane, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viegas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, X. Zheng, TensorFlow: large-scale machine learning on heterogeneous distributed systems, https://www.tensorflow.org/, 2015.

[67] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: an imperative style, high-performance deep learning library, arXiv:1912.01703, https://doi.org/10.48550/arXiv.1912.01703, Dec. 2019.

[68] R. Frostig, M.J. Johnson, C. Leary, Compiling machine learning programs via high-level tracing, http://github.com/jax-ml/jax, 2018.

[69] S. Thaler, G. Doehner, J. Zavadlav, Scalable Bayesian uncertainty quantification for neural network potentials: promise and pitfalls, J. Chem. Theory Comput. 19 (14) (2023) 4520–4532, https://doi.org/10.1021/acs.jctc.2c01267.

[70] S. Thaler, P. Fuchs, A. Cukarska, J. Zavadlav, JaxSGMC: modular stochastic gradient MCMC in JAX, SoftwareX 26 (2024) 101722, https://doi.org/10.1016/j.softx.2024.101722.

[71] S. Thaler, F. Mayr, S. Thomas, A. Gagliardi, J. Zavadlav, Active learning graph neural networks for partial charge prediction of metal-organic frameworks via dropout Monte Carlo, npj Comput. Mater. 10 (1) (2024) 1–10, https://doi.org/10.1038/s41524-024-01277-8.

[72] M.E. Tuckerman, Statistical Mechanics: Theory and Molecular Simulation, reprinted (with corr.) edition, Oxford Graduate Texts, Oxford Univ. Press, Oxford, 2015.

[73] M.E. Tuckerman, C.J. Mundy, G.J. Martyna, On the classical statistical mechanics of non-Hamiltonian systems, Europhys. Lett. 45 (2) (1999) 149, https://doi.org/10.1209/epl/i1999-00139-0.

[74] G.J. Martyna, M.L. Klein, M. Tuckerman, Nosé–Hoover chains: the canonical ensemble via continuous dynamics, J. Chem. Phys. 97 (4) (1992) 2635–2643, https://doi.org/10.1063/1.463940.

[75] G. Bussi, M. Parrinello, Accurate sampling using Langevin dynamics, Phys. Rev. E 75 (5) (2007) 056707, https://doi.org/10.1103/PhysRevE.75.056707.

[76] R.J. Sadus, Molecular simulation of orthobaric isochoric heat capacities near the critical point, Phys. Rev. E 99 (1) (2019) 012139, https://doi.org/10.1103/PhysRevE.99.012139.

[77] M. Braxenthaler, R. Unger, D. Auerbach, J.A. Given, J. Moult, Chaos in protein dynamics, Proteins, Struct. Funct. Bioinform. 29 (4) (1997) 417–425, https://doi.org/10.1002/(SICI)1097-0134(199712)29:4<417::AID-PROT2>3.0.CO;2-5.

[78] L. Metz, C.D. Freeman, S.S. Schoenholz, T. Kachman, Gradients are not all you need, arXiv:2111.05803, Jan. 2022.

[79] J. Ingraham, A. Riesselman, C. Sander, D. Marks, Learning protein structure with a differentiable simulator, in: International Conference on Learning Representations, 2019.

[80] B. Han, K. Yu, Refining potential energy surface through dynamical properties via differentiable molecular simulation, arXiv:2406.18269, Jun. 2024.

[81] R.W. Zwanzig, High-temperature equation of state by a perturbation method. I. Nonpolar gases, J. Chem. Phys. 22 (8) (2004) 1420–1426, https://doi.org/10.1063/1.1740409.

[82] G.M. Torrie, J.P. Valleau, Nonphysical sampling distributions in Monte Carlo free-energy estimation: umbrella sampling, J. Comput. Phys. 23 (2) (1977) 187–199, https://doi.org/10.1016/0021-9991(77)90121-8.

[83] M. Ceriotti, G.A.R. Brain, O. Riordan, D.E. Manolopoulos, The inefficiency of re-weighted sampling and the curse of system size in high-order path integration, Proc. R. Soc., Math. Phys. Eng. Sci. 468 (2137) (2012) 2–17, arXiv:1107.1908.

[84] S. Izvekov, G.A. Voth, A multiscale coarse-graining method for biomolecular systems, J. Phys. Chem. B 109 (7) (2005) 2469–2473, https://doi.org/10.1021/jp044629q.

[85] W. Wang, R. Gómez-Bombarelli, Coarse-graining auto-encoders for molecular dynamics, npj Comput. Mater. 5 (1) (2019) 1–9, https://doi.org/10.1038/s41524-019-0261-5.

[86] W.G. Noid, P. Liu, Y. Wang, J.-W. Chu, G.S. Ayton, S. Izvekov, H.C. Andersen, G.A. Voth, The multiscale coarse-graining method. II. Numerical implementation for coarse-grained molecular models, J. Chem. Phys. 128 (24) (2008) 244115, https://doi.org/10.1063/1.2938857.

[87] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, http://arxiv.org/abs/1412.6980, arXiv:1412.6980, https://doi.org/10.48550/arXiv.1412.6980, Jan. 2017.

[88] S. Stocker, J. Gasteiger, F. Becker, S. Günnemann, J.T. Margraf, How robust are modern graph neural network potentials in long and hot molecular dynamics simulations?, Mach. Learn.: Sci. Technol. 3 (4) (2022) 045010, https://doi.org/10.1088/2632-2153/ac9955.

[89] M. Welling, Y.W. Teh, Bayesian learning via stochastic gradient Langevin dynamics, in: International Conference on Machine Learning, 2011.

[90] M. Wen, E.B. Tadmor, Uncertainty quantification in molecular simulations with dropout neural network potentials, npj Comput. Mater. 6 (1) (2020) 1–10, https://doi.org/10.1038/s41524-020-00390-8.

[91] F. Musil, M.J. Willatt, M.A. Langovoy, M. Ceriotti, Fast and accurate uncertainty estimation in chemical machine learning, J. Chem. Theory Comput. 15 (2) (2019) 906–915, https://doi.org/10.1021/acs.jctc.8b00959.

[92] A. Das, H.C. Andersen, The multiscale coarse-graining method. V. Isothermal-isobaric ensemble, J. Chem. Phys. 132 (16) (2010) 164106, https://doi.org/10.1063/1.3394862.

[93] J.W. Wagner, J.F. Dama, A.E.P. Durumeric, G.A. Voth, On the representability problem and the physical meaning of coarse-grained models, J. Chem. Phys. 145 (4) (2016) 044108, https://doi.org/10.1063/1.4959168.

[94] O.T. Unke, M. Meuwly, PhysNet: a neural network for predicting energies, forces, dipole moments, and partial charges, J. Chem. Theory Comput. 15 (6) (2019) 3678–3693, https://doi.org/10.1021/acs.jctc.9b00181.

[95] S. Kullback, R.A. Leibler, On Information and Sufficiency, Ann. Math. Stat. 22 (1) (1951) 79–86.

[96] A. Das, H.C. Andersen, The multiscale coarse-graining method. III. A test of pairwise additivity of the coarse-grained potential and of new basis functions for the variational calculation, J. Chem. Phys. 131 (3) (2009) 034102, https://doi.org/10.1063/1.3173812.

[97] A.K. Soper, Empirical potential Monte Carlo simulation of fluid structure, Chem. Phys. 202 (2) (1996) 295–306, https://doi.org/10.1016/0301-0104(95)00357-6.

[98] J. Wang, S. Olsson, C. Wehmeyer, A. Pérez, N.E. Charron, G. de Fabritiis, F. Noé, C. Clementi, Machine learning of coarse-grained molecular dynamics force fields, ACS Cent. Sci. 5 (5) (2019) 755–767, https://doi.org/10.1021/acscentsci.8b00913.

[99] R. Ramakrishnan, P.O. Dral, M. Rupp, O.A. Von Lilienfeld, Big data meets quantum chemistry approximations: the Δ-machine learning approach, J. Chem. Theory Comput. 11 (5) (2015) 2087–2096.

[100] B. Lakshminarayanan, A. Pritzel, C. Blundell, Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles, Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017.

[101] C.H. Bennett, Efficient estimation of free energy differences from Monte Carlo data, J. Comput. Phys. 22 (2) (1976) 245–268, https://doi.org/10.1016/0021-9991(76)90078-4.

[102] M.A. Wyczalkowski, A. Vitalis, R.V. Pappu, New estimators for calculating solvation entropy and enthalpy and comparative assessments of their accuracy and precision, J. Phys. Chem. B 114 (24) (2010) 8166–8180, https://doi.org/10.1021/jp103050u.

[103] G. Imbalzano, Y. Zhuang, V. Kapil, K. Rossi, E.A. Engel, F. Grasselli, M. Ceriotti, Uncertainty estimation for molecular dynamics and sampling, J. Chem. Phys. 154 (2021) 074102, https://doi.org/10.1063/5.0036522.

[104] P. Ströker, R. Hellmann, K. Meier, Systematic formulation of thermodynamic properties in the N p T ensemble, Phys. Rev. E 103 (2) (2021) 023305, https://doi.org/10.1103/PhysRevE.103.023305.

[105] The HDF Group, Hierarchical data format, version 5, https://github.com/HDFGroup/hdf5.

[106] Y. Sugita, Y. Okamoto, Replica-exchange molecular dynamics method for protein folding, Chem. Phys. Lett. 314 (1–2) (1999) 141–151, https://doi.org/10.1016/S0009-2614(99)01123-9.

[107] A. Gil-Ley, G. Bussi, Enhanced conformational sampling using replica exchange with collective-variable tempering, J. Chem. Theory Comput. 11 (3) (2015) 1077–1085, https://doi.org/10.1021/ct5009087.

[108] L. Biewald, Experiment tracking with weights and biases, software available from https://www.wandb.com/, 2020.

[109] A. Chaimovich, M.S. Shell, Coarse-graining errors and numerical optimization using a relative entropy framework, J. Chem. Phys. 134 (9) (2011) 094112, https://doi.org/10.1063/1.3557038.

[110] J. Köhler, Y. Chen, A. Krämer, C. Clementi, F. Noé, Flow-matching: efficient coarse-graining of molecular dynamics without forces, J. Chem. Theory Comput. 19 (3) (2023) 942–952, https://doi.org/10.1021/acs.jctc.3c00016.

[111] T. Wen, R. Wang, L. Zhu, L. Zhang, H. Wang, D.J. Srolovitz, Z. Wu, Specialising neural network potentials for accurate properties and application to the mechanical response of titanium, npj Comput. Mater. 7 (1) (2021) 206.

[112] G. Simmons, H. Wang, Single Crystal Elastic Constants and Calculated Aggregate Properties, 2nd edition, M.I.T. Press, Cambridge, MA, 1971.